

LAPORAN PRAKTIKUM STRUKTUR DATA PEKAN 9
IMPLEMENTASI BINARY TREE DAN GRAPH TRAVERSAL



Disusun Oleh:
Annisa Layli Ramadhani
2511532024

Dosen Pengampu: Wahyudi. Dr.. S.T.M.T
Asisten Praktikum: Muhammad Galid Avero

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pertanggungjawaban atas praktikum yang telah dilakukan selama proses pembelajaran mata kuliah Struktur Data.

Laporan praktikum ini dibuat untuk membantu memahami konsep-konsep dasar Struktur Data serta penerapannya dalam pemrograman. Materi yang dibahas mencakup implementasi berbagai struktur data dan algoritma yang digunakan untuk menyelesaikan permasalahan dalam pengolahan data secara lebih efektif dan efisien. Dengan adanya praktikum ini, diharapkan mahasiswa dapat meningkatkan kemampuan logika, analisis, dan keterampilan pemrograman secara lebih baik.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penyusunan maupun isi materi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan laporan praktikum berikutnya.

Padang, 9 Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	5
1.1 Latar Belakang	5
1.2 Tujuan	5
1.3 Manfaat	6
BAB II PEMBAHASAN	7
2.1 Dasar Teori	7
2.1.1 Struktur Data <i>Tree</i>	7
2.1.2 <i>Binary Tree</i>	7
2.1.3 Struktur Data <i>Graph</i>	8
2.1.4 <i>Depth First Search</i>	8
2.1.5 <i>Breadth First Search</i>	9
2.2 Langkah Kerja.....	9
2.2.1 Kode Node_2511532024.....	9
2.2.2 Penjelasan Kode Node_2511532024	10
2.2.3 Kode BTree_2511532024	14
2.2.4 Penjelasan Kode BTree_2511532024.....	16
2.2.5 Kode BTreeDriver_2511532024.....	20
2.2.6 Penjelasan Kode BTreeDriver_2511532024	21
2.2.7 Kode GraphTraversal_2511532024.....	24
2.2.8 Penjelasan Kode GraphTraversal_2511532024	26
2.2.9 Output Kode Program.....	31
BAB III PENUTUP	32
3.1 Kesimpulan	32
DAFTAR PUSTAKA	33
TUGAS PRAKTIKUM STRUKTUR DATA	34
4.1 Soal Tugas	34
4.2 Kode program.....	34
4.2.1 Class PetaWisata_2522532024	34

4.3	Hasil dari Kode Program.....	45
4.3.1	Output Kode Program.....	45
4.3.2	Deskripsi Peta yang dibuat.....	45
4.3.3	Jumlah Node dan Edge.....	46
4.3.4	Hasil BFS	46
4.3.5	Hasil DFS	46
4.3.6	Kesimpulan perbandingan BFS dan DFS.....	47

BAB I PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep fundamental dalam ilmu komputer yang berperan penting dalam proses pengorganisasian, penyimpanan, dan pengelolaan data agar dapat diakses serta diproses secara efisien. Pemilihan struktur data yang tepat dapat meningkatkan kinerja suatu program, terutama dalam menangani data dalam jumlah besar dan permasalahan yang kompleks. Oleh karena itu, pemahaman mengenai berbagai jenis struktur data menjadi hal yang sangat penting bagi mahasiswa.

Salah satu struktur data yang banyak digunakan adalah *tree* dan *graph*. Struktur data *tree*, khususnya *Binary Tree*, memungkinkan data disusun secara hierarkis sehingga mempermudah proses pencarian, penyisipan, serta penelusuran data melalui metode traversal seperti *PreOrder*, *InOrder*, dan *PostOrder*. Sementara itu, struktur data *graph* digunakan untuk mempresentasikan hubungan antar objek yang saling terhubung, seperti jaringan komputer, media sosial, dan sistem navigasi.

Dalam pengoperasian *graph*, terdapat algoritma penelusuran yang umum digunakan, yaitu *Depth First Search* (DFS) dan *Breadth First Search* (BFS). Kedua algoritma tersebut memiliki karakteristik yang berbeda dalam menelusuri simpul-simpul pada *graph* dan banyak diterapkan dalam berbagai bidang, seperti pencarian jalur, analisis jaringan, serta pemecahan masalah berbasis graf.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut:

1. Memahami konsep dasar struktur data *Binary Tree* dan *Graph* beserta karakteristiknya.
2. Mempelajari implementasi *Binary Tree* menggunakan bahasa pemrograman Java.

3. Memahami proses penelusuran (traversal) pada *Binary Tree* menggunakan metode *PreOrder*, *InOrder*, dan *PostOrder*.
4. Mempelajari cara merepresentasikan *Graph* serta melakukan operasi dasar pada struktur data *Graph*.
5. Mengimplementasikan algoritma *Depth First Search* (DFS) dan *Breadth First Search* (BFS) untuk melakukan penelusuran pada *Graph*.

1.3 Manfaat

Manfaat dari pelaksanaan praktikum antara lain sebagai berikut:

1. Mahasiswa dapat memahami konsep dan prinsip kerja *Binary Tree* serta *Graph* secara lebih mendalam.
2. Mahasiswa memperoleh pengalaman dalam mengimplementasikan struktur data dan algoritma traversal menggunakan bahasa pemrograman Java.
3. Mahasiswa mampu membedakan penggunaan metode traversal pada *Binary Tree*, yaitu *PreOrder*, *InOrder*, dan *PostOrder*.
4. Mahasiswa dapat memahami perbedaan karakteristik dan penerapan algoritma DFS dan BFS dalam penelusuran *Graph*.
5. Mahasiswa dapat meningkatkan kemampuan berpikir logis, sistematis, dan analitis dalam menyelesaikan permasalahan komputasi.

BAB II

PEMBAHASAN

2.1 Dasar Teori

2.1.1 Struktur Data *Tree*

Tree merupakan struktur non-linear yang digunakan untuk menyimpan data dalam bentuk hierarki. Struktur ini terdiri dari atas sekumpulan simpul (*node*) yang saling terhubung melalui sisi (*edge*). *Node* paling atas disebut *root*, sedangkan *node* yang tidak memiliki anak disebut *leaf*. *Tree* banyak digunakan dalam berbagai aplikasi, seperti representasi struktur organisasi, sistem berkas (*file system*), dan basis data.

2.1.2 *Binary Tree*

Binary Tree jenis *tree* dimana setiap *node* memiliki paling banyak dua anak, yaitu *left child* (anak kiri) dan *right child* (anak kanan). *Binary Tree* memungkinkan proses pencarian dan pengolahan data dilakukan secara lebih terstruktur. Pada implementasinya, setiap *node* biasanya menyimpan data serta referensi ke *node* kiri dan kanan.

Beberapa metode traversal pada *Binary Tree* meliputi:

- *PreOrder Traversal (Root-Left-Right)*, yaitu mengunjungi *node* akar terlebih dahulu, kemudian *subtree* kiri, lalu *subtree* kanan.
- *InOrder Traversal (Left-Root-Right)*, yaitu mengunjungi *subtree* kiri, kemudian *node* akar, dan terakhir *subtree* kanan.
- *PostOrder Traversal (Left-Right-Root)*, yaitu mengunjungi *subtree* kiri, *subtree* kanan, kemudian *node* akar.

Traversal digunakan untuk menampilkan, mencari, atau memproses seluruh *node* dalam *tree* dengan urutan tertentu.

2.1.3 Struktur Data *Graph*

Graph merupakan data non-linear yang terdiri atas sekumpulan simpul (*vertex*) dan hubungan antar simpul yang disebut *edge*. *Graph* digunakan untuk merepresentasikan berbagai permasalahan yang melibatkan hubungan antar objek, seperti jaringan komputer, peta jalan, media sosial, dan sistem rekomendasi.

Berdasarkan arah sisinya, *graph* dibedakan menjadi:

- *Directed Graph (Digraph)*, yaitu *graph* yang memiliki arah pada setiap *edge*.
- *Undirected Graph*, yaitu *graph* yang tidak memiliki arah pada *edge* sehingga hubungan antar *vertex* bersifat dua arah.

Salah satu cara representasi *graph* adalah menggunakan *Adjacency List*, yaitu daftar yang menyimpan semua tetangga (*neighbor*) dari setiap *vertex*.

2.1.4 *Depth First Search*

Depth First Search (DFS) merupakan algoritma penelusuran *graph* yang bekerja dengan mengunjungi simpul sedalam mungkin sebelum kembali (*backtracking*) untuk menjelajahi simpul lainnya. DFS umumnya diimplementasikan menggunakan rekursi atau *stack*.

Algoritma DFS banyak digunakan dalam:

- Pencarian jalur pada *graph*.
- Deteksi siklus (*cycle detection*).
- *Topological sorting*.
- Menentukan komponen yang terhubung (*connected components*).

2.1.5 *Breadth First Search*

Breadth First Search (BFS) adalah algoritma penelusuran *graph* yang mengunjungi semua simpul pada tingkat yang sama terlebih dahulu sebelum berpindah ke tingkat berikutnya. BFS biasanya diimplementasikan menggunakan struktur data *queue* (antrian).

BFS memiliki beberapa penerapan, antara lain:

- Menentukan lintasan terpendek pada *graph* tak berbobot.
- Penelusuran jaringan komputer.
- Sistem navigasi dan pencarian rute.
- Analisis hubungan pada media sosial.

Pemahaman mengenai *Binary Tree*, *Graph*, serta algoritma DFS dan BFS sangat penting dalam pengembangan perangkat lunak karena konsep-konsep tersebut menjadi dasar dalam penyelesaian berbagai permasalahan komputasi secara efektif dan efisien.

2.2 Langkah Kerja

2.2.1 Kode Node_2511532024

```
1. public class Node_2511532024 {
2.     int data_2024;
3.     Node_2511532024 left_2024;
4.     Node_2511532024 right_2024;
5.
6.     public Node_2511532024(int data_2024) {
7.         this.data_2024 = data_2024;
8.         left_2024 = null;
9.         right_2024 = null;
10.    }
11.    public void setLeft_2024(Node_2511532024 node_2024)
12.    {
13.        if (left_2024 == null)
14.            left_2024 = node_2024;
15.    }
16.    public void setRight_2024(Node_2511532024 node_2024)
17.    {
18.        if (right_2024 == null)
19.            right_2024 = node_2024;
20.    }
21.    public Node_2511532024 getLeft_2024() {
22.        return left_2024;
23.    }
24. }
```

```

22. public Node_2511532024 getRight_2024() {
23.     return right_2024;
24. }
25. public int getData_2024() {
26.     return data_2024;
27. }
28. public void setData_2024(int data_2024) {
29.     this.data_2024 = data_2024;
30. }
31. void printPreOrder_2024(Node_2511532024 node_2024) {
32.     if(node_2024 == null)
33.         return;
34.     System.out.print(node_2024.data_2024 + " ");
35.     printPreOrder_2024(node_2024.left_2024);
36.     printPreOrder_2024(node_2024.right_2024);
37. }
38. void printPostOrder_2024(Node_2511532024 node_2024)
39. {
40.     if (node_2024 == null)
41.         return;
42.     printPostOrder_2024(node_2024.left_2024);
43.     printPostOrder_2024(node_2024.right_2024);
44.     System.out.print(node_2024.data_2024 + " ");
45. }
46. void printInOrder_2024(Node_2511532024 node_2024) {
47.     if (node_2024 == null)
48.         return;
49.     printInOrder_2024(node_2024.left_2024);
50.     System.out.print(node_2024.data_2024 + " ");
51.     printInOrder_2024(node_2024.right_2024);
52. }
53. public String print_2024() {
54.     return this.print_2024("", true, "");
55. }
56. public String print_2024(String prefix_2024, boolean
57. isTail_2024, String sb_2024) {
58.     if (right_2024 != null) {
59.         right_2024.print_2024(prefix_2024 +
60. (isTail_2024 ? "|" : " "), false, sb_2024);
61.     }
62.     System.out.println( prefix_2024+(isTail_2024 ?
63. "\\\-- " : "/-- ") + data_2024);
64.     if (left_2024 != null) {
65.         left_2024.print_2024(prefix_2024+(isTail_2024 ? "
66. " : "| "), true, sb_2024);
67.     }
68.     return sb_2024;
69. }
70. }
71. }

```

2.2.2 Penjelasan Kode Node_2511532024

1. Deklarasi Kelas dan Atribut

```
public class Node_2511532024 {
```

```
int data_2024;  
Node_2511532024 left_2024;  
Node_2511532024 right_2024;
```

Bagian ini mendefinisikan kelas *Node_2511532024* yang digunakan untuk merepresentasikan sebuah node pada struktur data *Binary Tree*. Variabel *data_2024* berfungsi untuk menyimpan nilai bertipe integer, sedangkan *left_2024* dan *right_2024* digunakan sebagai referensi yang menunjuk ke *node* anak kiri dan anak kanan.

2. Constructor Node

```
public Node_2511532024(int data_2024) {  
    this.data_2024 = data_2024;  
    left_2024 = null;  
    right_2024 = null;  
}
```

Constructor ini digunakan untuk membuat objek *node* baru. Parameter *data_2024* digunakan untuk menginisialisasi nilai data pada *node*. Sementara itu, *left_2024* dan *right_2024* diinisialisasi dengan nilai null karena *node* yang baru dibuat belum memiliki anak kiri maupun anak kanan.

3. Method Setter dan Getter

```
public void setLeft_2024(Node_2511532024 node_2024) {  
    if (left_2024 == null)  
        left_2024 = node_2024;  
}  
public void setRight_2024(Node_2511532024 node_2024) {  
    if (right_2024 == null)  
        right_2024 = node_2024;  
}  
public Node_2511532024 getLeft_2024() {
```

```

        return left_2024;
    }
    public Node_2511532024 getRight_2024() {
        return right_2024;
    }
    public int getData_2024() {
        return data_2024;
    }
    public void setData_2024(int data_2024) {
        this.data_2024 = data_2024;
    }
}

```

Method setLeft_2024() dan *setRight_2024()* digunakan untuk menetapkan *node* anak kiri dan kanan apabila posisi tersebut masih kosong. *Method getLeft_2024()* dan *getRight_2024()* digunakan untuk mengambil referensi *node* anak kiri dan kanan. *Method getData_2024()* berfungsi untuk memperoleh nilai data yang tersimpan pada *node*, sedangkan *setData_2024()* digunakan untuk mengubah nilai data pada *node*.

4. *Method PreOrder Traversal*

```

void printPreOrder_2024(Node_2511532024 node_2024) {
    if(node_2024 == null)
        return;
    System.out.print(node_2024.data_2024 + " ");
    printPreOrder_2024(node_2024.left_2024);
    printPreOrder_2024(node_2024.right_2024);
}

```

Method ini digunakan untuk melakukan *traversal* pohon menggunakan metode *PreOrder* dengan urutan *Root* → *Left* → *Right*. Pertama, data pada *node* saat ini ditampilkan, kemudian dilakukan penelusuran secara rekursif ke *subtree*

kiri dan *subtree* kanan. Jika *node* bernilai *null*, proses *traversal* dihentikan.

5. *Method PostOrder Traversal*

```
void printPostOrder_2024(Node_2511532024 node_2024) {  
    if (node_2024 == null)  
        return;  
    printPostOrder_2024(node_2024.left_2024);  
    printPostOrder_2024(node_2024.right_2024);  
    System.out.print(node_2024.data_2024 + " ");  
}
```

Method ini digunakan untuk melakukan *traversal* menggunakan metode *PostOrder* dengan urutan *Left* → *Right* → *Root*. Program akan menelusuri *subtree* kiri terlebih dahulu, dilanjutkan dengan *subtree* kanan, kemudian menampilkan data *node* saat ini.

6. *Method InOrder Traversal*

```
void printInOrder_2024(Node_2511532024 node_2024) {  
    if (node_2024 == null)  
        return;  
    printInOrder_2024(node_2024.left_2024);  
    System.out.print(node_2024.data_2024 + " ");  
    printInOrder_2024(node_2024.right_2024);  
}
```

Method ini digunakan untuk melakukan *traversal* menggunakan metode *InOrder* dengan urutan *Left* → *Root* → *Right*. Program akan menelusuri *subtree* kiri terlebih dahulu, kemudian menampilkan data *node* saat ini, dan dilanjutkan dengan menelusuri *subtree* kanan.

7. *Method Menampilkan Struktur Binary Tree*

```
public String print_2024() {
```

```

        return this.print_2024("", true, "");
    }
    public String print_2024(String prefix_2024, boolean
isTail_2024, String sb_2024) {
        if (right_2024 != null) {
            right_2024.print_2024(prefix_2024 + (isTail_2024 ? "|"
: "  "), false, sb_2024);
        }
        System.out.println(prefix_2024 + (isTail_2024 ? "\\-- " :
"/-- ") + data_2024);
        if (left_2024 != null) {
            left_2024.print_2024(prefix_2024 + (isTail_2024 ? "
: "|  "), true, sb_2024);
        }
        return sb_2024;
    }
}

```

Method print_2024() digunakan untuk menampilkan struktur *Binary Tree* dalam bentuk visual menyerupai pohon. *Method* ini bekerja secara rekursif dengan memanfaatkan parameter *prefix_2024* untuk mengatur indentasi dan *isTail_2024* untuk menentukan posisi cabang. Proses pencetakan dilakukan dengan menampilkan *subtree* kanan terlebih dahulu, kemudian *root*, dan terakhir *subtree* kiri sehingga struktur pohon dapat terlihat dengan lebih jelas.

2.2.3 Kode BTree_2511532024

```

1. public class BTree_2511532024 {
2.     private Node_2511532024 root_2024;
3.     private Node_2511532024 currentNode_2024;
4.     public BTree_2511532024() {
5.         root_2024 = null;
6.     }
7.     public boolean search_2024(int data_2024) {
8.         return search_2024(root_2024, data_2024);
9.     }
10.    private boolean search_2024(Node_2511532024
node_2024, int data_2024) {

```

```

11.         if (node_2024.getData_2024() == data_2024)
12.             return true;
13.         if (node_2024.getLeft_2024() != null)
14.             if
15.             (search_2024(node_2024.getLeft_2024(), data_2024))
16.                 return true;
17.         if (node_2024.getRight_2024() != null)
18.             if
19.             (search_2024(node_2024.getRight_2024(), data_2024))
20.                 return true;
21.         return false;
22.     }
23.     public void printInOrder_2024() {
24.         root_2024.printInOrder_2024(root_2024);
25.     }
26.     public void printPreOrder_2024() {
27.         root_2024.printPreOrder_2024(root_2024);
28.     }
29.     public void printPostOrder_2024() {
30.         root_2024.printPostOrder_2024(root_2024);
31.     }
32.     public Node_2511532024 getRoot_2024() {
33.         return root_2024;
34.     }
35.     public boolean isEmpty_2024() {
36.         return root_2024 == null;
37.     }
38.     public int countNodes_2024() {
39.         return countNodes_2024(root_2024);
40.     }
41.     private int countNodes_2024(Node_2511532024
42.         node_2024) {
43.         int count_2024 = 1;
44.         if (node_2024 == null) {
45.             return 0;
46.         } else {
47.             count_2024 +=
48.             countNodes_2024(node_2024.getLeft_2024());
49.             count_2024 +=
50.             countNodes_2024(node_2024.getRight_2024());
51.             return count_2024;
52.         }
53.     }
54.     public void print_2024() {
55.         root_2024.print_2024();
56.     }
57.     public Node_2511532024 getCurrent_2024() {
58.         return currentNode_2024;
59.     }
60.     public void setCurrent_2024(Node_2511532024
61.         node_2024) {

```

```

62.         this.currentNode_2024 = node_2024;
63.     }
64.
65.     public void setRoot_2024(Node_2511532024 root_2024)
66.     {
67.         this.root_2024 = root_2024;
68.     }

```

2.2.4 Penjelasan Kode BTree_2511532024

1. Deklarasi Kelas, Atribut, dan Constructor

```

public class BTree_2511532024 {
    private Node_2511532024 root_2024;
    private Node_2511532024 currentNode_2024;
    public BTree_2511532024() {
        root_2024 = null;
    }
}

```

Bagian ini mendefinisikan kelas *BTree_2511532024* yang digunakan untuk mengelola struktur data *Binary Tree*. Variabel *root_2024* digunakan untuk menyimpan *node* akar (*root*) dari *tree*, sedangkan *currentNode_2024* digunakan untuk menyimpan *node* yang sedang aktif atau sedang diproses. Constructor *BTree_2511532024()* menginisialisasi *tree* dalam keadaan kosong dengan menetapkan *root_2024* bernilai *null*.

2. Method Pencarian Data (Search)

```

public boolean search_2024(int data_2024) {
    return search_2024(root_2024, data_2024);
}

private boolean search_2024(Node_2511532024 node_2024,
int data_2024) {
    if (node_2024.getData_2024() == data_2024)
        return true;
    if (node_2024.getLeft_2024() != null)

```

```

        if (search_2024(node_2024.getLeft_2024(),
data_2024))
            return true;
        if (node_2024.getRight_2024() != null)
            if (search_2024(node_2024.getRight_2024(),
data_2024))
                return true;
        return false;
    }

```

Method search_2024() digunakan untuk mencari keberadaan suatu data dalam *Binary Tree*. *Method* publik memulai pencarian dari *node root*, sedangkan *method* privat melakukan proses pencarian secara rekursif. Algoritma akan memeriksa apakah data pada *node* saat ini sesuai dengan data yang dicari. Jika tidak ditemukan, proses pencarian dilanjutkan ke *subtree* kiri dan *subtree* kanan. *Method* akan mengembalikan nilai *true* apabila data ditemukan dan *false* apabila data tidak terdapat dalam *tree*.

3. *Method Traversal Binary Tree*

```

public void printInOrder_2024() {
    root_2024.printInOrder_2024(root_2024);
}
public void printPreOrder_2024() {
    root_2024.printPreOrder_2024(root_2024);
}
public void printPostOrder_2024() {
    root_2024.printPostOrder_2024(root_2024);
}

```

Ketiga *method* ini digunakan untuk melakukan *traversal* atau penelusuran seluruh *node* pada *Binary Tree*. *Method printInOrder_2024()* menampilkan *node* dengan urutan *Left-*

Root-Right, *printPreOrder_2024()* menggunakan urutan *Root-Left-Right*, sedangkan *printPostOrder_2024()* menggunakan urutan *Left-Right-Root*. Proses *traversal* dilakukan dengan memanggil *method* yang terdapat pada kelas *Node_2511532024*.

4. *Method* Pemeriksaan dan Perhitungan Jumlah *Node*

```
public boolean isEmpty_2024() {
    return root_2024 == null;
}

public int countNodes_2024() {
    return countNodes_2024(root_2024);
}

private int countNodes_2024(Node_2511532024
node_2024) {
    int count_2024 = 1;
    if (node_2024 == null) {
        return 0;
    } else {
        count_2024 +=
countNodes_2024(node_2024.getLeft_2024());
        count_2024 +=
countNodes_2024(node_2024.getRight_2024());
        return count_2024;
    }
}
```

Method isEmpty_2024() digunakan untuk memeriksa apakah *Binary Tree* masih kosong dengan mengecek apakah *root* bernilai *null*. Sementara itu, *method countNodes_2024()* digunakan untuk menghitung jumlah seluruh *node* yang terdapat dalam *tree* menggunakan teknik rekursif. Jika *node* bernilai *null*, *method* mengembalikan nilai 0. Jika *node* tidak

kosong, program akan menghitung *node* saat ini dan menjumlahkannya dengan total *node* pada *subtree* kiri dan *subtree* kanan.

5. *Method Menampilkan Struktur Tree*

```
public void print_2024() {  
    root_2024.print_2024();  
}
```

Method print_2024() digunakan untuk menampilkan struktur *Binary Tree* dalam bentuk visual seperti pohon. *Method* ini memanggil fungsi *print_2024()* yang terdapat pada kelas *Node_2511532024* untuk mencetak susunan *node* secara hierarkis.

6. *Method Getter dan Setter Root serta Current Node*

```
public Node_2511532024 getRoot_2024() {  
    return root_2024;  
}  
public Node_2511532024 getCurrent_2024() {  
    return currentNode_2024;  
}  
public void setCurrent_2024(Node_2511532024 node_2024)  
{  
    this.currentNode_2024 = node_2024;  
}  
public void setRoot_2024(Node_2511532024 root_2024) {  
    this.root_2024 = root_2024;  
}
```

Method getter digunakan untuk memperoleh nilai *root_2024* dan *currentNode_2024*, sedangkan *method setter* digunakan untuk mengubah atau menetapkan *node root* dan *current*

node. Method ini mempermudah pengelolaan *node* dalam *Binary Tree* dari kelas lain.

2.2.5 Kode BTreeDriver_2511532024

```
1. public class BTreeDriver_2511532024 {
2.     public static void main(String[] args) {
3.         // membuat pohon
4.         BTree_2511532024 tree_2024 = new
BTree_2511532024();
5.         System.out.print("Jumlah Simpul awal pohon:
");
6.
7.         System.out.println(tree_2024.countNodes_2024());
8.         // menambahkan simpul data 1
9.         Node_2511532024 root_2024 = new
Node_2511532024(1);
10.        // menjadikan simpul 1 sebagai root
11.        tree_2024.setRoot_2024(root_2024);
12.        System.out.print("Jumlah simpul jika hanya
ada root ");
13.
14.        System.out.println(tree_2024.countNodes_2024());
15.        Node_2511532024 node2 = new Node_2511532024
(2);
16.        Node_2511532024 node3 = new Node_2511532024
(3);
17.        Node_2511532024 node4 = new Node_2511532024
(4);
18.        Node_2511532024 node5 = new Node_2511532024
(5);
19.        Node_2511532024 node6 = new Node_2511532024
(6);
20.        Node_2511532024 node7 = new Node_2511532024
(7);
21.        Node_2511532024 node8 = new Node_2511532024
(8);
22.        Node_2511532024 node9 = new Node_2511532024
(9);
23.        root_2024.setLeft_2024(node2);
24.        node2.setLeft_2024(node4);
25.        node2.setRight_2024(node5);
26.        node4.setRight_2024(node8);
27.        root_2024.setRight_2024(node3);
28.        node3.setLeft_2024(node6);
29.        node3.setRight_2024(node7);
30.        node6.setLeft_2024(node9);
31.        // set root
32.
33.        tree_2024.setCurrent_2024(tree_2024.getRoot_2024());
34.        System.out.print("Menampilkan simpul terakhir:
");
35.
36.        System.out.println(tree_2024.getCurrent_2024().getDa
ta_2024());
```

```

33.     System.out.print("Jumlah simpul; setelah
        simpul 7 ditambahkan ");
34.
35.     System.out.println(tree_2024.countNodes_2024());
36.     System.out.println("InOrder: ");
37.     tree_2024.printInOrder_2024();
38.     System.out.println("\nPreOrder: ");
39.     tree_2024.printPreOrder_2024();
40.     System.out.println("\nPostOrder: ");
41.     tree_2024.printPostOrder_2024();
42.     System.out.println("\nMenampilkan simpul
        dalam bentuk pohon");
43.     tree_2024.print_2024();
44. }

```

2.2.6 Penjelasan Kode BTreeDriver_2511532024

1. Pembuatan Objek *Binary Tree* dan Pemeriksaan Awal

```

BTree_2511532024 tree_2024 = new BTree_2511532024();
System.out.print("Jumlah Simpul awal pohon: ");
System.out.println(tree_2024.countNodes_2024());

```

Bagian ini digunakan untuk membuat *objek tree_2024* dari kelas *BTree_2511532024* yang merepresentasikan sebuah *Binary Tree*. Setelah itu, program menampilkan jumlah simpul awal pada *tree* menggunakan *method countNodes_2024()*. Karena *tree* masih kosong, jumlah simpul yang ditampilkan adalah 0.

2. Pembuatan *Root* dan *Node Binary Tree*

```

Node_2511532024 root_2024 = new Node_2511532024(1);
tree_2024.setRoot_2024(root_2024);
System.out.print("Jumlah simpul jika hanya ada root ");
System.out.println(tree_2024.countNodes_2024());
Node_2511532024 node2 = new Node_2511532024(2);
Node_2511532024 node3 = new Node_2511532024(3);
Node_2511532024 node4 = new Node_2511532024(4);
Node_2511532024 node5 = new Node_2511532024(5);
Node_2511532024 node6 = new Node_2511532024(6);

```

```
Node_2511532024 node7 = new Node_2511532024(7);  
Node_2511532024 node8 = new Node_2511532024(8);  
Node_2511532024 node9 = new Node_2511532024(9);
```

Bagian ini digunakan untuk membuat *node-node* yang akan membentuk *Binary Tree*. *Node* dengan nilai 1 dijadikan sebagai *root* menggunakan *method setRoot_2024()*. Setelah *root* ditambahkan, program kembali menghitung jumlah simpul pada *tree*. Selanjutnya, dibuat *node-node* lain dengan nilai 2 hingga 9 yang nantinya akan dihubungkan untuk membentuk struktur pohon.

3. Penyusunan Struktur *Binary Tree*

```
root_2024.setLeft_2024(node2);  
node2.setLeft_2024(node4);  
node2.setRight_2024(node5);  
node4.setRight_2024(node8);  
  
root_2024.setRight_2024(node3);  
node3.setLeft_2024(node6);  
node3.setRight_2024(node7);  
node6.setLeft_2024(node9);
```

Bagian ini digunakan untuk menghubungkan setiap *node* sehingga membentuk struktur *Binary Tree*. *Node* 2 menjadi anak kiri dari *root*, sedangkan *node* 3 menjadi anak kanan *root*. *Node-node* lainnya juga dihubungkan sebagai anak kiri atau kanan dari *node* tertentu sehingga menghasilkan struktur pohon yang telah dirancang.

4. Pengaturan *Current Node* dan Menampilkan Informasi *Tree*

```
tree_2024.setCurrent_2024(tree_2024.getRoot_2024());  
  
System.out.print("Menampilkan simpul terakhir: ");
```

```
System.out.println(tree_2024.getCurrent_2024().getData_2024());
```

```
System.out.print("Jumlah simpul; setelah simpul 7  
ditambahkan ");
```

```
System.out.println(tree_2024.countNodes_2024());
```

Bagian ini digunakan untuk menetapkan *node root* sebagai *currentNode* menggunakan *method setCurrent_2024()*. Kemudian, program menampilkan data dari *current node* yang sedang aktif. Setelah seluruh *node* ditambahkan, program menghitung kembali jumlah total simpul yang terdapat pada *Binary Tree*.

5. Traversal Binary Tree

```
System.out.println("InOrder: ");  
tree_2024.printInOrder_2024();
```

```
System.out.println("\nPreOrder: ");  
tree_2024.printPreOrder_2024();
```

```
System.out.println("\nPostOrder: ");  
tree_2024.printPostOrder_2024();
```

Bagian ini digunakan untuk menampilkan hasil *traversal Binary Tree* menggunakan tiga metode *traversal* yang berbeda, yaitu:

- *InOrder (Left - Root - Right)*, digunakan untuk mengunjungi *subtree* kiri terlebih dahulu, kemudian *root*, lalu *subtree* kanan.
- *PreOrder (Root - Left - Right)*, digunakan untuk mengunjungi *root* terlebih dahulu, kemudian *subtree* kiri dan *subtree* kanan.

- *PostOrder (Left - Right - Root)*, digunakan untuk mengunjungi *subtree* kiri, *subtree* kanan, dan terakhir *root*. *Traversal* ini berguna untuk memahami berbagai cara penelusuran data pada *Binary Tree*.

6. Menampilkan Struktur *Binary Tree*

```
System.out.println("\nMenampilkan simpul dalam bentuk
pohon");
tree_2024.print_2024();
```

Bagian ini digunakan untuk menampilkan struktur *Binary Tree* secara visual dalam bentuk pohon. *Method print_2024()* akan mencetak susunan *node* beserta hubungan antar *node* sehingga memudahkan pengguna dalam memahami struktur *tree* yang telah dibuat.

2.2.7 Kode GraphTraversal_2511532024

```
1. public class GraphTraversal_2511532024 {
2.     private Map<String, List<String>> graph_2024 = new
    HashMap<>();
3.
4.     // menambahkan edge (graf tak berarah)
5.     public void addEdge_2024(String node1, String node2)
    {
6.         graph_2024.putIfAbsent(node1, new
        ArrayList<>());
7.         graph_2024.putIfAbsent(node2, new
        ArrayList<>());
8.         graph_2024.get(node1).add(node2);
9.         graph_2024.get(node2).add(node1);
10.    }
11.    // menampilkan graf awal
12.    public void printGraph_2024() {
13.        System.out.println("Graf Awal (Adjacency
        List): ");
14.        for (String node_2024 : graph_2024.keySet())
        {
15.            System.out.print(node_2024 + " -> ");
16.            List<String> neighbors_2024 =
            graph_2024.get(node_2024);
17.            System.out.println(String.join(", ",
            neighbors_2024));
18.        }
19.        System.out.println();
20.    }
```

```

21.
22. //DFS rekursif
23. public void dfs_2024(String start_2024) {
24.     Set<String> visited_2024 = new HashSet<>();
25.     System.out.println("Penelusuran DFS: ");
26.     dfsHelper_2024(start_2024, visited_2024);
27.     System.out.println();
28. }
29. private void dfsHelper_2024(String current_2024,
30. Set<String> visited_2024) {
31.     if(visited_2024.contains(current_2024))
32.         return;
33.     visited_2024.add(current_2024);
34.     System.out.print(current_2024 + " ");
35.     for (String neighbor_2024 :
36. graph_2024.getDefault(current_2024, new
37. ArrayList<>())) {
38.         dfsHelper_2024(neighbor_2024,
39. visited_2024);
40.     }
41. }
42. // BFS iteratif
43. public void bfs_2024(String start_2024) {
44.     Set<String> visited_2024 = new HashSet<>();
45.     Queue<String> queue_2024 = new LinkedList<>();
46.     queue_2024.add(start_2024);
47.     visited_2024.add(start_2024);
48.     System.out.println("Penelusuran BFS: ");
49.     while (!queue_2024.isEmpty()) {
50.         String current_2024 = queue_2024.poll();
51.         System.out.print(current_2024 + " ");
52.         for (String neighbor_2024 :
53. graph_2024.getDefault(current_2024, new
54. ArrayList<>())) {
55.             if(!visited_2024.contains(neighbor_2024)) {
56.                 queue_2024.add(neighbor_2024);
57.                 visited_2024.add(neighbor_2024);
58.             }
59.         }
60.     }
61.     System.out.println();
62. }
63. // main
64. public static void main(String[] args) {
65.     GraphTraversal_2511532024 graph_2024 = new
66. GraphTraversal_2511532024();
67.     // contoh graf: A-B, A-C, B-D, B-E
68.     graph_2024.addEdge_2024("A", "B");
69.     graph_2024.addEdge_2024("A", "C");
70.     graph_2024.addEdge_2024("B", "D");
71.     graph_2024.addEdge_2024("B", "E");
72.     // cetak garf awal

```

```

68.     System.out.println("Graf Awal adalah: ");
69.     graph_2024.printGraph_2024();
70.     //lakukan penelusuran
71.     graph_2024.dfs_2024("A");
72.     graph_2024.bfs_2024("A");
73. }
74. }

```

2.2.8 Penjelasan Kode GraphTraversal_2511532024

1. Deklarasi Kelas dan Struktur Data *Graph*

```

public class GraphTraversal_2511532024 {
    private Map<String, List<String>> graph_2024 = new
    HashMap<>();

```

Bagian ini mendefinisikan kelas *GraphTraversal_2511532024* yang digunakan untuk mengimplementasikan operasi pada struktur data *graph*. Variabel *graph_2024* menggunakan *HashMap* dengan tipe *Map<String, List<String>>* untuk merepresentasikan *graph* dalam bentuk *Adjacency List*, di mana setiap simpul (*vertex*) memiliki daftar simpul tetangga (*neighbors*) yang terhubung dengannya.

2. Method Penambahan *Edge* dan Menampilkan *Graph*

```

public void addEdge_2024(String node1, String node2) {
    graph_2024.putIfAbsent(node1, new ArrayList<>());
    graph_2024.putIfAbsent(node2, new ArrayList<>());
    graph_2024.get(node1).add(node2);
    graph_2024.get(node2).add(node1);
}

public void printGraph_2024() {
    System.out.println("Graf Awal (Adjacency List): ");
    for (String node_2024 : graph_2024.keySet()) {
        System.out.print(node_2024 + " -> ");
        List<String> neighbors_2024 =
graph_2024.get(node_2024);

```

```
        System.out.println(String.join(", ", neighbors_2024));
    }
    System.out.println();
}
```

Method addEdge_2024() digunakan untuk menambahkan hubungan (*edge*) antar dua simpul pada *graph* tak berarah (*undirected graph*). *Method putIfAbsent()* memastikan bahwa setiap simpul telah tersedia dalam *graph* sebelum hubungan ditambahkan. Karena *graph* bersifat tak berarah, setiap simpul akan saling ditambahkan ke daftar tetangga masing-masing.

Method printGraph_2024() digunakan untuk menampilkan representasi *graph* dalam bentuk *adjacency list*. Program akan menampilkan setiap simpul beserta seluruh simpul yang terhubung dengannya.

3. *Method Traversal Depth First Search (DFS)*

```
public void dfs_2024(String start_2024) {
    Set<String> visited_2024 = new HashSet<>();
    System.out.println("Penelusuran DFS: ");
    dfsHelper_2024(start_2024, visited_2024);
    System.out.println();
}

private void dfsHelper_2024(String current_2024,
    Set<String> visited_2024) {
    if(visited_2024.contains(current_2024))
        return;

    visited_2024.add(current_2024);
    System.out.print(current_2024 + " ");
}
```

```

        for (String neighbor_2024 :
graph_2024.getDefault(current_2024, new
ArrayList<>())) {
            dfsHelper_2024(neighbor_2024, visited_2024);
        }
    }
}

```

Method dfs_2024() digunakan untuk melakukan penelusuran *graph* menggunakan algoritma *Depth First Search (DFS)* yang dimulai dari simpul tertentu. *Method* ini membuat *HashSet* bernama *visited_2024* untuk menyimpan simpul yang telah dikunjungi, kemudian memanggil *method* rekursif *dfsHelper_2024()*.

Method dfsHelper_2024() melakukan proses DFS secara rekursif. Jika simpul saat ini sudah pernah dikunjungi, proses akan dihentikan untuk mencegah terjadinya perulangan tak hingga. Jika belum dikunjungi, simpul akan ditandai sebagai telah dikunjungi dan ditampilkan. Selanjutnya, program akan menelusuri seluruh simpul tetangga hingga mencapai simpul terdalam sebelum kembali ke simpul sebelumnya (*backtracking*).

4. *Method Traversal Breadth First Search (BFS)*

```

public void bfs_2024(String start_2024) {
    Set<String> visited_2024 = new HashSet<>();
    Queue<String> queue_2024 = new LinkedList<>();

    queue_2024.add(start_2024);
    visited_2024.add(start_2024);

    System.out.println("Penelusuran BFS: ");

    while (!queue_2024.isEmpty()) {

```

```

String current_2024 = queue_2024.poll();
System.out.print(current_2024 + " ");

for (String neighbor_2024 :
graph_2024.getDefault(current_2024, new
ArrayList<>())) {
    if(!visited_2024.contains(neighbor_2024)) {
        queue_2024.add(neighbor_2024);
        visited_2024.add(neighbor_2024);
    }
}
}

System.out.println();
}

```

Method bfs_2024() digunakan untuk melakukan penelusuran *graph* menggunakan algoritma *Breadth First Search (BFS)*. *Method* ini memanfaatkan *HashSet* untuk menyimpan simpul yang telah dikunjungi dan *Queue (LinkedList)* untuk mengatur urutan penelusuran.

Proses BFS dimulai dengan memasukkan simpul awal ke dalam antrian dan menandainya sebagai telah dikunjungi. Selama antrian tidak kosong, program akan mengambil simpul terdepan, menampilkannya, kemudian memeriksa seluruh simpul tetangga yang belum dikunjungi untuk dimasukkan ke dalam antrian. Dengan cara ini, BFS akan menelusuri *graph* berdasarkan tingkat kedekatan simpul (*level by level*).

5. *Method* Utama (*Main*)

```

public static void main(String[] args) {
    GraphTraversal_2511532024 graph_2024 = new

```

```
GraphTraversal_2511532024();
```

```
graph_2024.addEdge_2024("A", "B");
```

```
graph_2024.addEdge_2024("A", "C");
```

```
graph_2024.addEdge_2024("B", "D");
```

```
graph_2024.addEdge_2024("B", "E");
```

```
System.out.println("Graf Awal adalah: ");
```

```
graph_2024.printGraph_2024();
```

```
graph_2024.dfs_2024("A");
```

```
graph_2024.bfs_2024("A");
```

```
}
```

Method main() berfungsi sebagai titik awal eksekusi program. Pada bagian ini dibuat objek *graph_2024* dari kelas *GraphTraversal_2511532024*. Selanjutnya, program membangun *graph* dengan menambahkan beberapa *edge*, yaitu antara simpul A-B, A-C, B-D, dan B-E.

Setelah *graph* terbentuk, program menampilkan representasi *graph* menggunakan *adjacency list* melalui *method printGraph_2024()*. Kemudian, dilakukan penelusuran *graph* menggunakan algoritma *Depth First Search (DFS)* dan *Breadth First Search (BFS)* dengan titik awal penelusuran dari simpul A.

2.2.9 Output Code Program

1. Output Code Program BTreeDriver_2511532024

```
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root 1
Menampilkan simpul terakhir: 1
Jumlah simpul; setelah simpul 7 ditambahkan 9
InOrder:
4 8 2 5 1 9 6 3 7
PreOrder:
1 2 4 8 5 3 6 9 7
PostOrder:
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
|
|      /-- 7
|      /-- 3
|      |      \-- 6
|      |      \-- 9
|-- 1
|
|      /-- 5
|-- 2
|
|      /-- 8
|-- 4
```

2. Output Code Program GraphTraversal_2511532024

```
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS: |
A B C D E
```

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data *Binary Tree* dan *Graph* merupakan struktur data non-linear yang memiliki peranan penting dalam pengelolaan dan penelusuran data. Implementasi *Binary Tree* memungkinkan penyimpanan data secara hierarkis serta mendukung proses *traversal* menggunakan metode *PreOrder*, *InOrder*, dan *PostOrder* untuk mengakses seluruh simpul dalam urutan tertentu.

Selain itu, implementasi *Graph* menggunakan representasi *Adjacency List* mempermudah pengelolaan hubungan antar simpul dalam suatu jaringan. Algoritma *Depth First Search (DFS)* dan *Breadth First Search (BFS)* dapat digunakan untuk melakukan penelusuran graph dengan karakteristik yang berbeda, di mana DFS menelusuri simpul hingga kedalaman tertentu sebelum kembali ke simpul sebelumnya, sedangkan BFS menelusuri simpul berdasarkan tingkat kedekatan dari simpul awal.

Melalui praktikum ini, mahasiswa memperoleh pemahaman mengenai konsep, implementasi, serta cara kerja *Binary Tree* dan *Graph* menggunakan bahasa pemrograman Java. Selain itu, praktikum ini juga meningkatkan kemampuan analisis, logika pemrograman, dan keterampilan dalam menerapkan algoritma *traversal* untuk menyelesaikan permasalahan yang berkaitan dengan struktur data.

DAFTAR PUSTAKA

GeeksforGeeks. (2025, August 21). *BFS vs DFS for binary tree*. GeeksforGeeks.

<https://www.geeksforgeeks.org/dsa/bfs-vs-dfs-binary-tree/>

GeeksforGeeks. (2025, October 25). *Depth first search or DFS for a graph*.

GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>

GeeksforGeeks. (2025, December 8). *Level order traversal (Breadth First Search) of binary tree*. GeeksforGeeks.

<https://www.geeksforgeeks.org/dsa/level-order-tree-traversal/>

GeeksforGeeks. (2025, December 15). *Binary tree data structure*. GeeksforGeeks.

<https://www.geeksforgeeks.org/dsa/binary-tree-data-structure/>

TUGAS PRAKTIKUM STRUKTUR DATA

4.1 Soal Tugas

Mahasiswa diminta untuk mengimplementasikan algoritma *BFS* (*Breadth First Search*) dan *DFS* (*Depth First Search*) menggunakan bahasa Java dengan antarmuka GUI. Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk *Graph*.

Setiap mahasiswa wajib membuat peta lokasi yang berbeda dengan mahasiswa lain. Peta dapat berupa: Gedung Perkuliahan, Fakultas, Universitas, Sekolah, Rumah Sakit, Mall, Tempat Wisata, Bandara, Atau lokasi lainnya. Program harus mampu: Menampilkan *graph* dalam bentuk visual, Menentukan titik awal (*Start*), Menentukan titik tujuan (*Goal*), Melakukan pencarian menggunakan *BFS*, Melakukan pencarian menggunakan *DFS*, Menampilkan jalur yang ditemukan, Menampilkan urutan simpul yang dikunjungi, Urutan *node* yang dikunjungi, Jumlah *node* yang dieksplorasi

4.2 Kode program

4.2.1 Class PetaWisata_2522532024

```
1. public class PetaWisata_2511532024 extends JFrame {
2.
3.     private HashMap<String, List<String>> graph_2024;
4.     private HashMap<String, Point> posisiNode_2024;
5.
6.     private JComboBox<String> cmbStart_2024;
7.     private JComboBox<String> cmbGoal_2024;
8.
9.     private JButton btnBFS_2024;
10.    private JButton btnDFS_2024;
11.    private JButton btnReset_2024;
12.
13.    private JTextArea txtHasil_2024;
14.
15.    private GraphPanel_2024 panelGraph_2024;
16.
17.    private List<String> jalur_2024;
18.    private List<String> nodeDikunjungi_2024;
19.    private Set<String> visitedNodes_2024;
20.
21.    private int jumlahNodeDikunjungi_2024;
```

```

22.
23.     public PetaWisata_2511532024() {
24.
25.         setTitle("PENCARIAN JALUR MENGGUNAKAN BFS
DAN DFS");
26.         setSize(950, 700);
27.         setLocationRelativeTo(null);
28.
29.         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
30.         setLayout(new BorderLayout(10, 10));
31.
32.         graph_2024 = new HashMap<>();
33.         posisiNode_2024 = new HashMap<>();
34.
35.         jalur_2024 = new ArrayList<>();
36.         nodeDikunjungi_2024 = new ArrayList<>();
37.         visitedNodes_2024 = new HashSet<>();
38.
39.         buatGraph_2024();
40.         buatGUI_2024();
41.
42.         setVisible(true);
43.     }
44.
45.     private void buatGraph_2024() {
46.
47.         tambahEdge_2024("Pantai Padang", "Jembatan
Siti Nurbaya");
48.         tambahEdge_2024("Pantai Padang", "Museum
Adityawarman");
49.         tambahEdge_2024("Pantai Padang", "Masjid
Raya Sumbar");
50.
51.         tambahEdge_2024("Jembatan Siti Nurbaya",
"Gunung Padang");
52.         tambahEdge_2024("Jembatan Siti Nurbaya",
"Pantai Air Manis");
53.
54.         tambahEdge_2024("Pantai Air Manis", "Pantai
Nirwana");
55.         tambahEdge_2024("Pantai Nirwana", "Pelabuhan
Teluk Bayur");
56.
57.         tambahEdge_2024("Museum Adityawarman", "Kota
Tua Padang");
58.         tambahEdge_2024("Kota Tua Padang", "Taman
Budaya");
59.
60.         tambahEdge_2024("Taman Budaya", "Masjid Raya
Sumbar");
61.         tambahEdge_2024("Masjid Raya Sumbar",
"Museum Adityawarman");
62.
63.         tambahEdge_2024("Gunung Padang", "Pantai
Nirwana");
64.         tambahEdge_2024("Gunung Padang", "Kota Tua
Padang");

```

```

64.
65.         tambahEdge_2024("Pelabuhan Teluk Bayur",
66.         "Pantai Air Manis");
67.         tambahEdge_2024("Taman Budaya", "Pantai
68.         Padang");
69.         posisiNode_2024.put("Pantai Padang", new
70.         Point(120, 70));
71.         posisiNode_2024.put("Jembatan Siti Nurbaya",
72.         new Point(300, 70));
73.         posisiNode_2024.put("Museum Adityawarman",
74.         new Point(480, 70));
75.         posisiNode_2024.put("Masjid Raya Sumbar",
76.         new Point(660, 70));
77.         posisiNode_2024.put("Pantai Air Manis", new
78.         Point(120, 180));
79.         posisiNode_2024.put("Gunung Padang", new
80.         Point(300, 180));
81.         posisiNode_2024.put("Kota Tua Padang", new
82.         Point(480, 180));
83.         posisiNode_2024.put("Pantai Nirwana", new
84.         Point(660, 180));
85.         posisiNode_2024.put("Taman Budaya", new
86.         Point(250, 290));
87.         posisiNode_2024.put("Pelabuhan Teluk Bayur",
88.         new Point(550, 290));
89.     }
90.
91.     private void tambahEdge_2024(String asal_2024,
92.     String tujuan_2024) {
93.         graph_2024.putIfAbsent(asal_2024, new
94.         ArrayList<>());
95.         graph_2024.putIfAbsent(tujuan_2024, new
96.         ArrayList<>());
97.         graph_2024.get(asal_2024).add(tujuan_2024);
98.         graph_2024.get(tujuan_2024).add(asal_2024);
99.     }
100.
101.     private void buatGUI_2024() {
102.         JPanel panelAtas_2024 = new JPanel(new
103.         BorderLayout());
104.         panelAtas_2024.setBackground(new Color(44,
105.         62, 80));
106.         JLabel lblJudul_2024 = new JLabel("PENCARIAN
107.         JALUR MENGGUNAKAN BFS DAN DFS",
108.         SwingConstants.CENTER);
109.         lblJudul_2024.setForeground(Color.WHITE);
110.         lblJudul_2024.setFont(new Font("Arial",
111.         Font.BOLD, 22));
112.         panelAtas_2024.setBorder(BorderFactory.createEmptyBo
113.         rder(15, 10, 15, 10));
114.         panelAtas_2024.add(lblJudul_2024,
115.         BorderLayout.CENTER);
116.         add(panelAtas_2024, BorderLayout.NORTH);
117.     }

```

```

98.         JPanel panelKontrol_2024 = new JPanel(new
GridBagLayout());
99.         panelKontrol_2024.setBorder(BorderFactory.createTitl
edBorder("PENGATURAN  PENCARIAN"));
100.
101.         GridBagConstraints gbc_2024 = new
GridBagConstraints();
102.
103.         gbc_2024.insets = new Insets(5, 5, 5,
5);
104.         gbc_2024.anchor =
GridBagConstraints.WEST;
105.
106.         JLabel lblStart_2024 = new
JLabel("Lokasi Awal :");
107.         lblStart_2024.setFont(new
Font("Arial", Font.BOLD, 14));
108.         cmbStart_2024 = new
JComboBox<>(graph_2024.keySet().toArray(new
String[0]));
109.
110.         cmbStart_2024.setPreferredSize(new
Dimension(200, 30));
111.         JLabel lblGoal_2024 = new
JLabel("Lokasi Tujuan :");
112.         lblGoal_2024.setFont( new
Font("Arial", Font.BOLD, 14));
113.
114.         cmbGoal_2024 = new
JComboBox<>(graph_2024.keySet().toArray( new
String[0]));
115.
116.         cmbGoal_2024.setPreferredSize(new
Dimension(200, 30));
117.
118.         btnBFS_2024 = new JButton("BFS");
119.         btnBFS_2024.setBackground(new
Color(46, 204, 113));
120.
121.         btnBFS_2024.setForeground( Color.WHITE);
122.         btnBFS_2024.setFont(new Font("Arial",
Font.BOLD, 13));
123.         btnDFS_2024 = new JButton("DFS");
124.         btnDFS_2024.setBackground(new
Color(243, 156, 18));
125.         btnDFS_2024.setForeground(Color.WHITE);
126.         btnDFS_2024.setFont(new Font("Arial",
Font.BOLD, 13));
127.         btnReset_2024 = new JButton("RESET");
128.         btnReset_2024.setBackground(new
Color(231, 76, 60));
129.         btnReset_2024.setForeground(Color.WHITE);
130.         btnReset_2024.setFont(new
Font("Arial", Font.BOLD, 13));

```

```

130.
131.         gbc_2024.gridx = 0;
132.         gbc_2024.gridy = 0;
133.         panelKontrol_2024.add(lblStart_2024,
            gbc_2024);
134.         gbc_2024.gridx = 1;
135.         panelKontrol_2024.add(cmbStart_2024,
            gbc_2024);
136.         gbc_2024.gridx = 0;
137.         gbc_2024.gridy = 1;
138.         panelKontrol_2024.add(lblGoal_2024,
            gbc_2024);
139.         gbc_2024.gridx = 1;
140.         panelKontrol_2024.add(cmbGoal_2024,
            gbc_2024);
141.         gbc_2024.gridx = 2;
142.         gbc_2024.gridy = 0;
143.         panelKontrol_2024.add(btnBFS_2024,
            gbc_2024);
144.         gbc_2024.gridx = 3;
145.         panelKontrol_2024.add(btnDFS_2024,
            gbc_2024);
146.         gbc_2024.gridx = 4;
147.         panelKontrol_2024.add(btnReset_2024,
            gbc_2024);
148.
149.         panelGraph_2024 = new
            GraphPanel_2024();
150.         panelGraph_2024.setPreferredSize( new
            Dimension(700, 350));
151.         panelGraph_2024.setBorder(BorderFactory.createTitled
            Border("VISUALISASI GRAPH"));
152.
153.         txtHasil_2024 = new JTextArea(8, 40);
154.         txtHasil_2024.setEditable(false);
155.         txtHasil_2024.setFont(new
            Font("Monospaced", Font.PLAIN, 14));
156.         txtHasil_2024.setText("Hasil
            Pencarian :\n" + "Jalur :\n" + "Node Dikunjungi :\n"
            +
157.             "Jumlah Node Dikunjungi : 0");
158.
159.         JScrollPane scrollHasil_2024 = new
            JScrollPane(txtHasil_2024);
160.         scrollHasil_2024.setBorder(BorderFactory.createTitle
            dBorder("HASIL PENCARIAN"));
161.         JPanel panelTengah_2024 =new
            JPanel(new BorderLayout());
162.         panelTengah_2024.add(panelKontrol_2024,
            BorderLayout.NORTH);
163.         panelTengah_2024.add(panelGraph_2024,
            BorderLayout.CENTER);
164.         panelTengah_2024.add(scrollHasil_2024,
            BorderLayout.SOUTH);

```

```

165.         add(panelTengah_2024,
166.             BorderLayout.CENTER);
167.         btnBFS_2024.addActionListener(e -> {
168.             String start_2024 =
169.                 cmbStart_2024.getSelectedItem().toString();
170.             String goal_2024 =
171.                 cmbGoal_2024.getSelectedItem().toString();
172.             BFS_2024(start_2024, goal_2024);
173.         });
174.         btnDFS_2024.addActionListener(e -> {
175.             String start_2024 =
176.                 cmbStart_2024.getSelectedItem().toString();
177.             String goal_2024 =
178.                 cmbGoal_2024.getSelectedItem().toString();
179.             DFS_2024(start_2024, goal_2024);
180.         });
181.         btnReset_2024.addActionListener(e ->
182.         {
183.             resetGraph_2024();
184.         });
185.         private void BFS_2024(String start_2024,
186.             String goal_2024) {
187.             Queue<String> queue_2024 = new
188.                 LinkedList<>();
189.             HashSet<String> visited_2024 = new
190.                 HashSet<>();
191.             HashMap<String, String> parent_2024 =
192.                 new HashMap<>();
193.             nodeDikunjungi_2024.clear();
194.             visitedNodes_2024.clear();
195.             jalur_2024.clear();
196.             queue_2024.offer(start_2024);
197.             visited_2024.add(start_2024);
198.             boolean ditemukan_2024 = false;
199.             while (!queue_2024.isEmpty()) {
200.                 String current_2024 =
201.                     queue_2024.poll();
202.                 nodeDikunjungi_2024.add(current_2024);
203.                 visitedNodes_2024.add(current_2024);
204.                 if
205.                     (current_2024.equals(goal_2024)) {
206.                         ditemukan_2024 = true;
207.                         break;
208.                     }
209.                 for (String tetangga_2024 :
210.                     graph_2024.get(current_2024)) {

```

```

207.         if
208.         (!visited_2024.contains(tetangga_2024)) {
209.             visited_2024.add(tetangga_2024);
210.             parent_2024.put(tetangga_2024, current_2024);
211.             queue_2024.offer(tetangga_2024);
212.         }
213.     }
214.
215.     jumlahNodeDikunjungi_2024 =
        nodeDikunjungi_2024.size();
216.     if (ditemukan_2024) {
217.         String current_2024 = goal_2024;
218.         while (current_2024 != null) {
219.             jalur_2024.add(current_2024);
220.             current_2024 =
                parent_2024.get(current_2024);
221.         }
222.
223.         Collections.reverse(jalur_2024);
224.     }
225.
226.     displayPath_2024("BFS");
227.
228.     panelGraph_2024.repaint();
229. }
230.
231. private void DFS_2024(String start_2024,
    String goal_2024) {
232.     Stack<String> stack_2024 = new
        Stack<>();
233.     HashSet<String> visited_2024 = new
        HashSet<>();
234.     HashMap<String, String> parent_2024 =
        new HashMap<>();
235.
236.     nodeDikunjungi_2024.clear();
237.     visitedNodes_2024.clear();
238.     jalur_2024.clear();
239.
240.     stack_2024.push(start_2024);
241.     boolean ditemukan_2024 = false;
242.
243.     while (!stack_2024.isEmpty()) {
244.         String current_2024 =
            stack_2024.pop();
245.         if
            (visited_2024.contains(current_2024)) {
246.             continue;
247.         }
248.
249.         visited_2024.add(current_2024);

```

```

250.     nodeDikunjungi_2024.add(current_2024);
251.     visitedNodes_2024.add(current_2024);
252.         if
253.         (current_2024.equals(goal_2024)) {
254.             ditemukan_2024 = true;
255.             break;
256.         }
257.         List<String> tetangga_2024 =
258.         graph_2024.get(current_2024);
259.         for (int i_2024 =
260.         tetangga_2024.size() - 1; i_2024 >= 0; i_2024--) {
261.             String next_2024 =
262.             tetangga_2024.get(i_2024);
263.             if
264.             (!visited_2024.contains(next_2024)) {
265.                 parent_2024.put(next_2024, current_2024);
266.                 stack_2024.push(next_2024);
267.             }
268.         }
269.         jumlahNodeDikunjungi_2024 =
270.         nodeDikunjungi_2024.size();
271.         if (ditemukan_2024) {
272.             String current_2024 = goal_2024;
273.             while (current_2024 != null) {
274.                 jalur_2024.add(current_2024);
275.                 current_2024 =
276.                 parent_2024.get(current_2024);
277.             }
278.             Collections.reverse(jalur_2024);
279.         }
280.         displayPath_2024("DFS");
281.         panelGraph_2024.repaint();
282.     }
283.
284.     private void displayPath_2024(String
285.     metode_2024) {
286.         StringBuilder hasil_2024 = new
287.         StringBuilder();
288.         hasil_2024.append("Metode :
289.         ").append(metode_2024).append("\n\n");
290.         if (jalur_2024.isEmpty()) {
291.             hasil_2024.append("Jalur : Tidak
292.             ditemukan\n\n");

```

```

290.         } else { hasil_2024.append("Jalur :
");
291.         hasil_2024.append(String.join(" →
", jalur_2024));
292.         hasil_2024.append("\n\n");
293.     }
294.     hasil_2024.append("Node
Dikunjungi :\n");
295.     hasil_2024.append(String.join(" → ",
nodeDikunjungi_2024));
296.     hasil_2024.append("\n\nJumlah Node
Dieksplorasi : ");
297.     hasil_2024.append(jumlahNodeDikunjungi_2024);
298.     txtHasil_2024.setText(hasil_2024.toString());
299.     }
300.
301.     private void resetGraph_2024() {
302.         jalur_2024.clear();
303.         nodeDikunjungi_2024.clear();
304.         visitedNodes_2024.clear();
305.         jumlahNodeDikunjungi_2024 = 0;
306.         txtHasil_2024.setText("Hasil
Pencarian :\n" + "Jalur :\n" + "Node Dikunjungi :\n"
+ "Jumlah Node Dikunjungi : 0");
307.         panelGraph_2024.repaint();
308.     }
309.
310.     class GraphPanel_2024 extends JPanel {
311.         @Override
312.         protected void
paintComponent(Graphics g_2024) {
313.             super.paintComponent(g_2024);
314.             Graphics2D g2d_2024 = (Graphics2D)
g_2024;
315.             int graphWidth_2024 = 600;
316.             int graphHeight_2024 = 300;
317.             int offsetX_2024 = (getWidth() -
graphWidth_2024) / 2;
318.             int offsetY_2024 = (getHeight() -
graphHeight_2024) / 2;
319.             g2d_2024.setRenderingHint(RenderingHints.KEY_ANTIALI
ASING, RenderingHints.VALUE_ANTIALIAS_ON);
320.             g2d_2024.setStroke( new
BasicStroke(2));
321.
322.             // Menggambar edge
323.             for (String node_2024 :
graph_2024.keySet()) {
324.                 Point p1_2024 =
posisiNode_2024.get(node_2024);
325.                 for (String tetangga_2024 :
graph_2024.get( node_2024)) {
326.                     Point p2_2024 =
posisiNode_2024.get(tetangga_2024);

```

```

327.
328.         g2d_2024.setColor(Color.GRAY);
329.         g2d_2024.drawLine(p1_2024.x + offsetX_2024,
330.                             p1_2024.y + offsetY_2024,
331.                             p2_2024.x +
332.                             offsetX_2024, p2_2024.y + offsetY_2024);
333.             }
334.         }
335.         // Highlight jalur hasil
336.         pencarian
337.         if (jalur_2024.size() > 1) {
338.             g2d_2024.setStroke(new
339.             BasicStroke(4));
340.             for (int i_2024 = 0; i_2024 <
341.             jalur_2024.size() - 1; i_2024++) {
342.                 Point p1_2024 =
343.                 posisiNode_2024.get(jalur_2024.get(i_2024));
344.                 Point p2_2024 =
345.                 posisiNode_2024.get(jalur_2024.get(i_2024 + 1));
346.                 g2d_2024.setColor(new
347.                 Color(46, 204, 113));
348.                 g2d_2024.drawLine(p1_2024.x + offsetX_2024,
349.                                     p1_2024.y + offsetY_2024,
350.                                     p2_2024.x +
351.                                     offsetX_2024, p2_2024.y + offsetY_2024);
352.             }
353.         }
354.         // Menggambar node
355.         for (String node_2024 :
356.         posisiNode_2024.keySet()) {
357.             Point point_2024 =
358.             posisiNode_2024.get(node_2024);
359.             if
360.             (jalur_2024.contains(node_2024)) {
361.                 g2d_2024.setColor(new
362.                 Color(46, 204, 113));
363.             }
364.             else if
365.             (visitedNodes_2024.contains(node_2024)) {
366.                 g2d_2024.setColor(new
367.                 Color(52, 152, 219));
368.             } else {
369.                 g2d_2024.setColor(new
370.                 Color(236, 240, 241));
371.             }
372.         }
373.         int lebarNode_2024 = 130;
374.         int tinggiNode_2024 = 40;
375.         g2d_2024.fillRoundRect(point_2024.x + offsetX_2024 -
376.         (lebarNode_2024 / 2),

```

```

362.                point_2024.y +
offsetY_2024 - (tinggiNode_2024 / 2), lebarNode_2024,
363.                tinggiNode_2024, 20, 20);
364.
365.
366.        g2d_2024.setColor(Color.BLACK);
367.        g2d_2024.drawRoundRect(point_2024.x + offsetX_2024 -
(lebarNode_2024 / 2),
368.                point_2024.y +
offsetY_2024 - (tinggiNode_2024 / 2), lebarNode_2024,
369.                tinggiNode_2024, 20,
20);
370.
371.                g2d_2024.setFont(new
Font("Arial", Font.BOLD, 11));
372.                FontMetrics fm_2024 =
g2d_2024.getFontMetrics();
373.                int lebarTeks_2024 =
fm_2024.stringWidth(node_2024);
374.                int tinggiTeks_2024 =
fm_2024.getAscent();
375.
376.        g2d_2024.drawString( node_2024, point_2024.x +
offsetX_2024 - (lebarTeks_2024 / 2),
377.                point_2024.y +
offsetY_2024 + (tinggiTeks_2024 / 4));
378.    }
379.
380.    // Keterangan warna
381.    g2d_2024.setFont(new Font("Arial",
Font.PLAIN, 12));
382.    g2d_2024.setColor(new Color(236,
240, 241));
383.    g2d_2024.fillRect(20, 20, 15, 15);
384.    g2d_2024.setColor(Color.BLACK);
385.    g2d_2024.drawRect(20, 20, 15, 15);
386.    g2d_2024.drawString("Belum
Dikunjungi", 45, 33);
387.    g2d_2024.setColor(new Color(52,
152, 219));
388.    g2d_2024.fillRect(20, 45, 15, 15);
389.    g2d_2024.setColor(Color.BLACK);
390.    g2d_2024.drawRect(20, 45, 15, 15);
391.    g2d_2024.drawString("Sudah
Dikunjungi", 45, 58);
392.    g2d_2024.setColor(new Color(46,
204, 113));
393.    g2d_2024.fillRect(20, 70, 15, 15);
394.    g2d_2024.setColor(Color.BLACK);
395.    g2d_2024.drawRect(20, 70, 15, 15);
396.    g2d_2024.drawString("Jalur
Ditemukan", 45, 83);
397.    }
398.

```

```

399.         public static void main(String[] args) {
400.             SwingUtilities.invokeLater(() -> {
401.                 new PetaWisata_2511532024();
402.             });
403.         }
404.     }

```

4.3 Hasil dari Kode Program

4.3.1 Output Kode Program

- BFS



- DFS



4.3.2 Deskripsi Peta yang dibuat

Peta yang dibuat merepresentasikan beberapa lokasi tempat wisata di Kota Padang yang dihubungkan dalam bentuk *graph* tidak berarah (*undirected graph*). Setiap *node* pada *graph* merepresentasikan satu lokasi wisata, sedangkan *edge* menunjukkan hubungan atau jalur antar lokasi tersebut.

Contoh lokasi yang digunakan:

- Pantai Padang

- Jembatan Siti Nurbaya
- Museum Adityawarman
- Gunung Padang
- Pantai Air Manis
- Pantai Nirwana
- Taman Budaya
- Kota Tua Padang
- Pelabuhan Teluk Bayur
- Masjid Raya Sumbar

4.3.3 Jumlah *Node* dan *Edge*

Jumlah *Node* : 10 *Node* (lokasi wisata)

Jumlah *Edge* : 15 *Edge* (hubungan antar lokasi)

Edge dibuat secara dua arah (*undirected graph*), sehingga setiap hubungan dapat dilalui bolak-balik

4.3.4 Hasil BFS

- *Start* : Pantai Padang
- *Goal* : Pantai Nirwana

Urutan node dikunjungi :

Pantai Padang → Jembatan Siti Nurbaya → Museum Adityawarman → Masjid Raya Sumbar → Pantai Air Manis → Kota Tua Padang → Pantai Nirwana

Jalur BFS :

Pantai Padang → Jembatan Siti Nurbaya → Pantai Air Manis → Pantai Nirwana

4.3.5 Hasil DFS

- *Start* : Pantai Padang
- *Goal* : Pantai Nirwana

Urutan node dikunjungi :

Pantai Padang → Jembatan Siti Nurbaya → Gunung Padang
→ Pantai Nirwana

Jalur DFS :

Pantai Padang → Jembatan Siti Nurbaya → Gunung Padang
→ Pantai Nirwana

4.3.6 Kesimpulan perbandingan BFS dan DFS

Berdasarkan hasil implementasi, dapat disimpulkan bahwa:

- *BFS (Breadth First Search)* lebih efektif untuk mencari jalur terpendek karena menelusuri *node* secara bertahap berdasarkan level.
- *DFS (Depth First Search)* lebih cepat mencapai suatu jalur tertentu, tetapi tidak menjamin jalur yang ditemukan adalah jalur terpendek.
- BFS lebih stabil untuk pencarian jalur optimal, sedangkan DFS lebih efisien dalam eksplorasi kedalam *graph*.

Dengan demikian, pemilihan algoritma bergantung pada kebutuhan, apakah mengutamakan jalur terpendek (BFS) atau eksplorasi cepat (DFS).

.